

1) ArrayBasics.java

Initially

marks = [0, 0, 0, 0, 0]

names = [null, null, null]

mark.length = 5

a = [10, 20, 30, 40, 0, 0, 0, 0] size = 4

 $\therefore a[2] = 30$ insert 15 at 1st index

size = 4, shift = 0

i = 4 $\rightarrow a[4] = a[3] \rightarrow 40$ shift 1i = 3 $\rightarrow a[3] = a[2] \rightarrow 30$ shift 2i = 2 $\rightarrow a[2] = a[1] \rightarrow 20$ shift 3

i = 1 (condition false)

a[1] = 15

 $\therefore a = [10, 15, 20, 30, 40, 0, 0, 0]$

size = 5 shift = 3

insert 50 at last [size = 5, index = 5]

i = 5 $\rightarrow 5 > 5$ false

a[5] = 50

a = [10, 15, 20, 30, 40, 50, 0, 0] size = 6, shift = 0

Delete index 0

size = 6, index = 0

i = 0 $\rightarrow a[0] = a[1] = 15$ i = 1 $\rightarrow a[1] = a[2] = 20$ i = 2 $\rightarrow a[2] = a[3] = 30$

$i=3 \rightarrow a[3] = a[4] \rightarrow 40$

$i=4 \rightarrow a[4] = a[5] \rightarrow 50$

$\therefore \text{shifts} = 5 \quad a[5] = 0 \quad \text{size} = 5$
 $a = [15, 20, 30, 40, 50, 0, 0, 0]$

if $a[8] \rightarrow$ exception (ArrayIndexOutOfBoundsException)

$r = 0 \rightarrow [1]$

Pascal Triangle

$r = 1 \rightarrow [1, 1]$

$r = 2 \rightarrow [1, 2, 1]$

$r = 3 \rightarrow [1, 3, 3, 1]$

$a.\text{length} = 8 \quad , \quad a.\text{length} * 2 = 16$

bigger = $[15, 20, 30, 40, 50, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$

2> Stack ArrayIntDemo.java

operation	stack	Top	O/p
initial	empty	-1	—
push 10	10	0	10
push 20	10, 20	1	20 10
peek	10, 20	1	20 10 Top = 20
Display	10, 20	1	20 10

operation	stack	top	output
Push 30	10, 20, 30	2	30 20 10
push 40	10, 20, 30, 40	3	40 30 20 10
push 50	10, 20, 30, 40	3	overflow
pop	10, 20, 30	2	30 20 10
peek	10, 20, 30	2	Top = 30
exit	10, 20, 30	2	

3) Stack Demo - Generic <T>

• integer stack

max = 10 top = -1

Push(10) := top = 0 & stack[0] = 10

push(20) := top = 1 & stack[1] = 20

pop(20) : top = 1 & stack[1] = 20

pop() : stack[1] = 20, top = 1

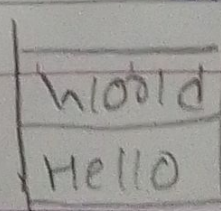
return 20

for string

max = 100 top = -1

push("Hello") : top = 0 stack[0] = "Hello"

push("World") : top = 1 stack[1] = "World"



pop() :

Stack[1] = "World"

top = 0 return "World"

• isBalanced ("{[()]}")

expression = "[()]"

length = 6

max = 6 top = 1 stack = empty

i	ch	operation	top	olp
initial	-	-	-1	empty
0	{	push {	0	{
1	[	push [	1	{[
2	(	push (	2	{[(
3	)	pop) → match	1	{[
4	]	pop]	0	{
5	}	pop }	-1	empty

top == 1

isEmpty()

top == -1

-1 == -1 true

{[()]} = Balanced

Q.4) Stack Demo.java

max = 100 stack [char]

stack = char[100] top = -1

expression = "{[()]}"

expression.length() = 6

index	0	1	2	3	4	5
	{	[	(	)	]	}

isBalanced()

top = -1 stack = empty

i = 0

ch = '{' push('{')

top = -1

++top = 0 stack[0] = '{'

top = 0

i = 1

ch = '[' push('[')

top = 0 ++top = 1

stack[1] = '[' top = 1

i = 2

ch = '(' push('(')

top = 1 ++top → 2

stack[2] = '(' top = 2

$i = 3$

$ch = ')'$

$isEmpty(): top == -1$

$2 == -1 \rightarrow false$

$open = pop()$

$open = stack[2] = 'C'$

$top -- \rightarrow 1$

$check \rightarrow isMatchingPair('C', ')') \rightarrow true$

$i = 4$

$ch = ']'$

$isEmpty(): 1 == -1 \rightarrow false$

$open = pop()$

$open = stack[1] = '['$

$top -- \rightarrow 0$

$isMatchingPair('[', ']') \rightarrow true$

$i = 5$

$ch = '}'$

$isEmpty(): 0 == -1 \rightarrow false$

$open = pop()$

$open = '{'$

$top -- \rightarrow -1$

$isMatchingPair('{', '}') \rightarrow true$

$top == -1$

$-1 == -1 \rightarrow true$

$\therefore isBalanced("[[C()]]") = true$

⑤ Expression Evaluation Demo.java

i/p = 2 3 1 * + 9 -

expression.trim().split(" ")

[2, 3, 1, *, +, 9, -]

stack = empty

step	i/p	operation	stack
			empty
1	2	push 2	[2]
2	3	push 3	[2, 3]
3	1	push 1	[2, 3, 1]
4	*	pop 1, pop 3 $\rightarrow 3 * 1 = 3$ push 3	[2, 3]
5	+	pop 3, pop 2 $\rightarrow$ $2 + 3 = 5$ push 5	[5]
6	9	push 9	[5, 9]
7	-	pop 9, pop 5 $\rightarrow 5 - 9 = -4$ $\rightarrow$ push -4	[-4]

i/p = 2 not operator

push(2)

i/p 3 not operator

push(3)

i/p 1 not -1-

push(1)

i/p * $\therefore$ pop 1 & 3

$1 * 3 = 3$ push(3)

1
3
2

3
2

$i/p \quad + \quad := \quad pop \quad 3 \quad 2$
 $3 + 2 = 5$
 $push \quad 5$

5

$i/p \quad 9 \quad : \quad push(9)$

9
5

$i/p \quad - \quad :- \quad pop \quad 9 \quad 5$

$5 - 9 = -4$

$push(-4)$

-4

6> Parenthesis Match Demo

initially stack = Empty

• $(a+b) * (c+d)$

$(\rightarrow push \rightarrow [($

$) \rightarrow pop(\rightarrow match \rightarrow []$

$(\rightarrow push \rightarrow [($

$) \rightarrow pop(\rightarrow match \rightarrow []$

final stack = empty

result = Balanced

• $\{[(c)]\}$

$\{ \rightarrow push \rightarrow [\{$

$[\rightarrow push \rightarrow [\{, [$

$(\rightarrow push \rightarrow [\{, [, ($

$) \rightarrow pop(\rightarrow match \rightarrow [\{, [$

$] \rightarrow pop[\rightarrow match \rightarrow [\{$

$\} \rightarrow pop\{ \rightarrow match \rightarrow []$

Balanced

• $(a+b)*c)$

$(\rightarrow \text{push} \rightarrow [()]$

$) \rightarrow \text{pop } (\rightarrow \text{not matching return false}$
not Balanced

• $((xy))$

$(\rightarrow \text{push} \rightarrow [()]$

$(\rightarrow \text{push} \rightarrow [(, (]$

$) \rightarrow \text{pop }) \rightarrow \text{match} \rightarrow [(]$

loop end

stack $[(]$ isEmpty = false
not balanced

7) ReverseStringStackDemo.java

inp = "HELLO"

length = 5

stack = empty

• input.charAt(i)

$i = 0$ push(H)

$i = 1$ push(E)

$i = 2$ push(L)

$i = 3$ push(L)

$i = 4$ push(O)

[H]

[H, E]

[H, E, L]

[H, E, L, L]

[H, E, L, L, O]

O
L
L
E
H

pop() = "O"

pop() = "L"

pop() = "L"

pop() = "E"

pop() = "H"

[H, E, L, L]

[H, E, L]

[H, E]

[H]

[]

OIP

O

OL

OLL

OLLE

OLLEH

8) StackUsingLinkedListInt Demo.java

top = null size = 0 stack empty

push 10

newNode.data = 10

newNode.next = null

top = newNode

size = 1

Stack : Top → [10 | null]

push 20

newNode.data = 20

newNode.next = top

size = 2

Stack : Top [20 | .] → [10 | null]

Peek

top != null → true

top.data = 20

stack unchanged

Display

current.data = 20

current = current.next

current.data = 10

current = null

O/P 20 10

size = 2

Pop

top != null → true

value = top.data = 20

size -- ; ∴ size = 1

g) StackUsingLinkedList.java

operations	stack	top	size of stack
initially	empty	null	0
push 10	10	10	1
push 20	20, 10	20	2
push 30	30, 20, 10	30	3
Display	30, 20, 10	30	3
peek	30, 20, 10	30	3
pop	20, 10	20	2
size	20, 10	20	2
Display	20, 10	20	2

10) UndoOperationDemo.java

choice	operation	stack	o/p
	Initial	empty	
1	add open file	open file	saved
2	add edit file	open file, edit file	saved
13	add save file	open, edit, save	saved
2	undo	open, edit	save unda
3	show	open, edit	Display

Queue

1. QueueUsingStackDemo.java

Initially

stackNewest = []

top = -1

Queue = empty

stackOldest = []

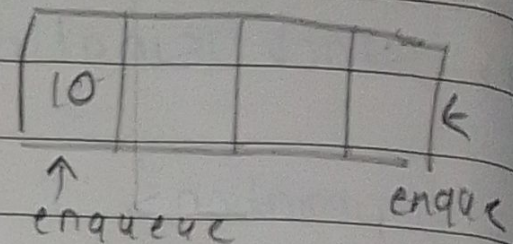
top = -1

• enqueue 10

stackNewest.push(10)

top = 0

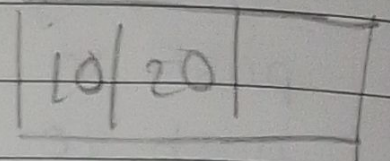
Queue = 10



• enqueue 20

top = 1 stackNewest = [10, 20]

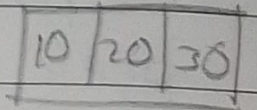
Queue = 10, 20



• enqueue 30

top = 2 stackNewest = [10, 20, 30]

Queue = 10, 20, 30



• Peek

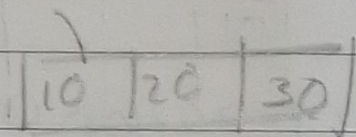
stackOldest = []

stackOldest = [30, 20, 10]

top = 2

olp = 10

top



• Display (Pop from stackOldest)

olp → 10, 20, 30

• Dequeue

stackOldest = [30, 20]

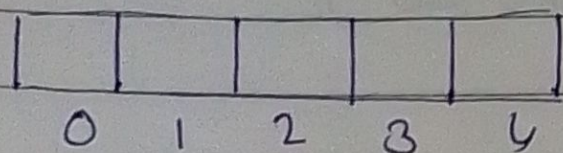
size = 0

2. Queue Demo. java

max = 5

queue []

queue = empty



front = -1 rear = -1

• enqueue (10)

rear = max - 1

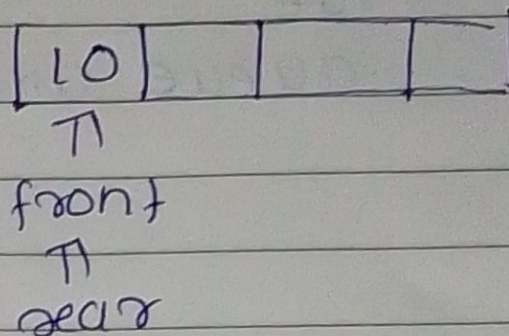
 $-1 = 4$ false

front == -1 true

front = 0

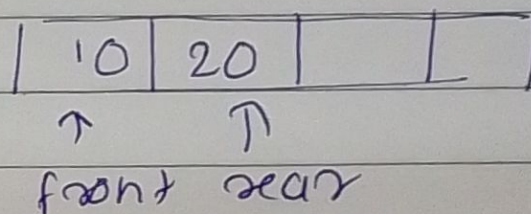
 $0 + \text{rear} \therefore \text{rear} = 0$

front = 0



• enqueue (20)

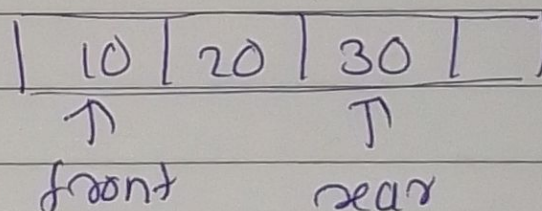
front = 0

 $\text{rear} = 0 + 1 = 1$ 

• enqueue (30)

front = 0

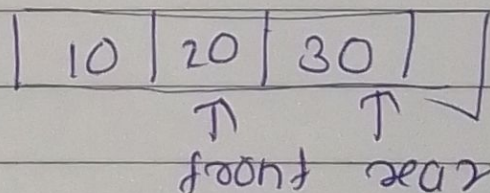
rear = 2



• dequeue

front = 1

rear = 2



• dequeue

front = 2

rear = 2

